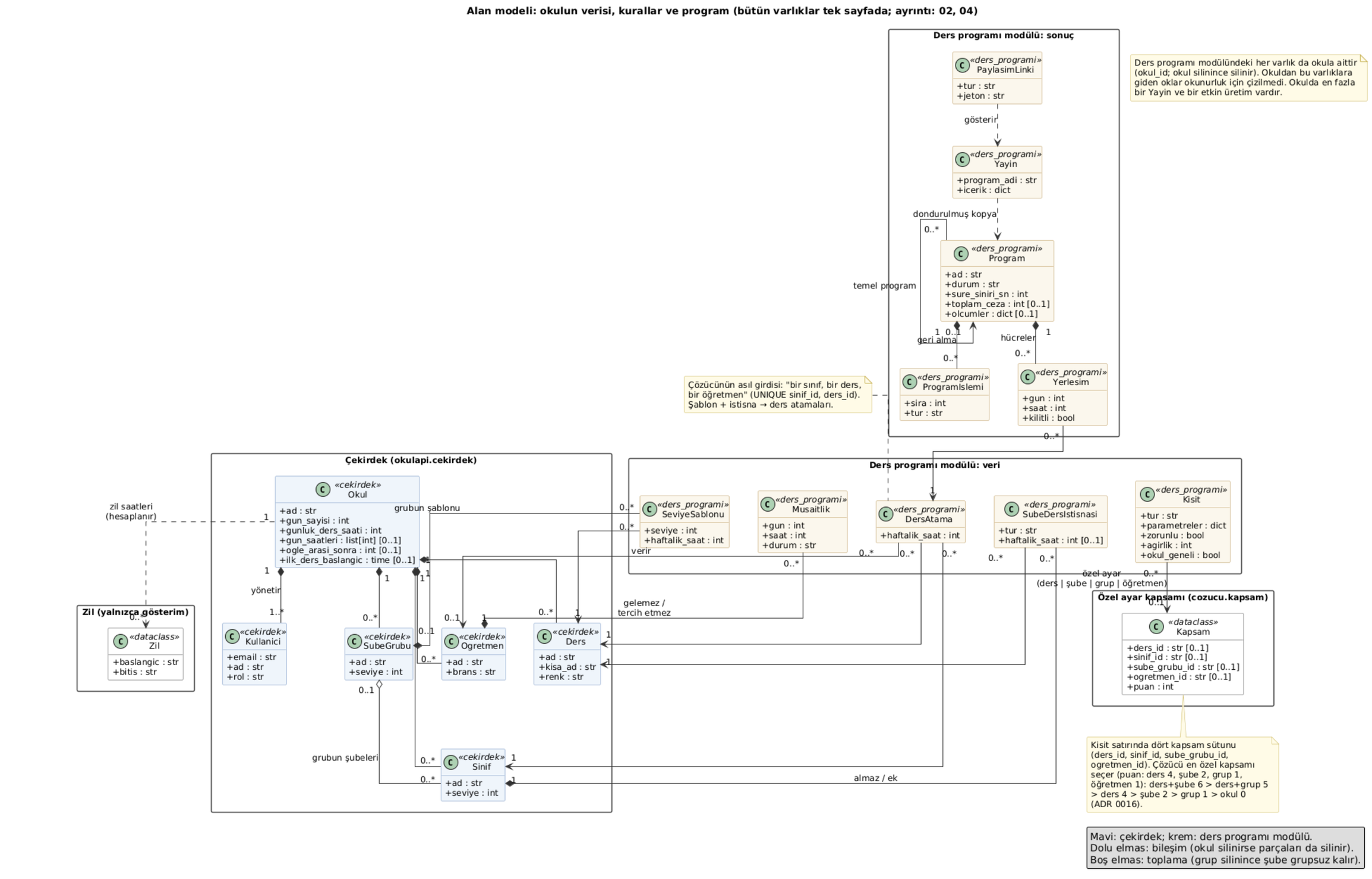
Çizelge – Okul Ders Programı Oluşturma Sistemi

UML diyagramları (sınıf, nesne, dağıtım, paket)

Kodun son hâlinden üretildi: 24.10.2026. Adlar api/tests/test_diyagramlar.py ile kodla denetlenir.

Sayfa 2   11 Sınıf diyagramı: Alan modeli
Sayfa 3   02 Sınıf diyagramı: Çekirdek modeller ve okul izolasyonu
Sayfa 4   04 Sınıf diyagramı: Ders programı modülünün tabloları
Sayfa 5   06 Sınıf diyagramı: Çözücü
Sayfa 6   12 Sınıf diyagramı: API katmanları, kuyruk ve çıktı
Sayfa 7   03 Sınıf diyagramı: Çekirdek katmanları ve modül sözleşmesi
Sayfa 8   05 Sınıf diyagramı: Ders programı modülünün katmanları
Sayfa 9   10 Sınıf diyagramı: Kimlik doğrulama
Sayfa 10   01 Paket diyagramı: Genel bakış
Sayfa 11   07 Nesne diyagramı: Örnek okuldan bir kesit (veri girişi)
Sayfa 12   08 Nesne diyagramı: Program üretildikten sonra
Sayfa 13   09 Dağıtım diyagramı: Docker Compose

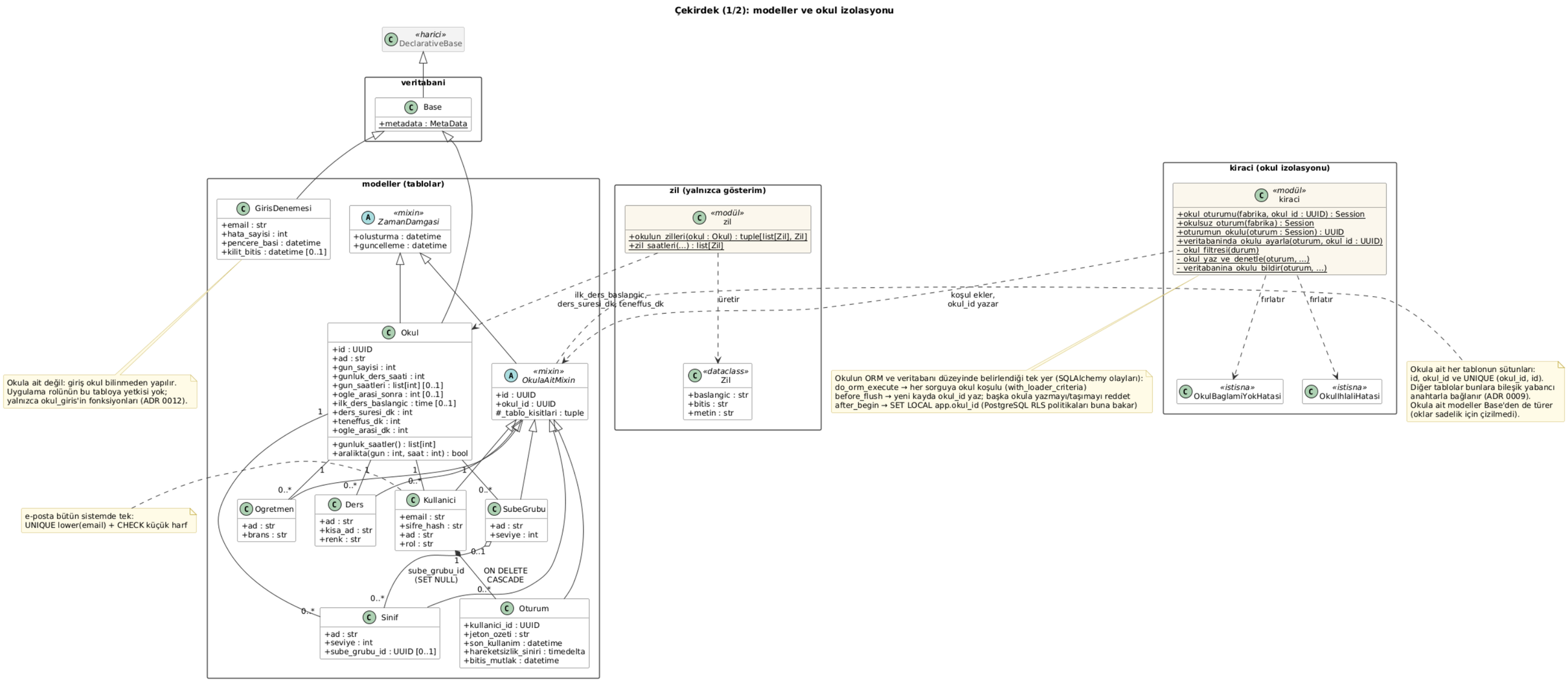
11 Sınıf diyagramı: Alan modeli



Okulun bütün verisi tek sayfada: çekirdekteki okul, kullanıcı, öğretmen, şube, şube grubu ve ders; ders programı modülündeki şablon, şube istisnası, ders ataması, müsaitlik, kural (özel ayar ve kapsamı), program, yerleşim, elle düzeltme işlemi, yayın ve paylaşım linki. Çokluklar veritabanı kısıtlarından gelir: örneğin her (şube, ders) çifti için tek ders ataması vardır ve bir kuralın özel ayarı ders, şube, grup ya da öğretmene bağlanır. Zil saatleri saklanmaz, okul ayarlarından hesaplanır.

11-alan-modeli.png · kaynak: docs/diyagramlar/11-alan-modeli.puml

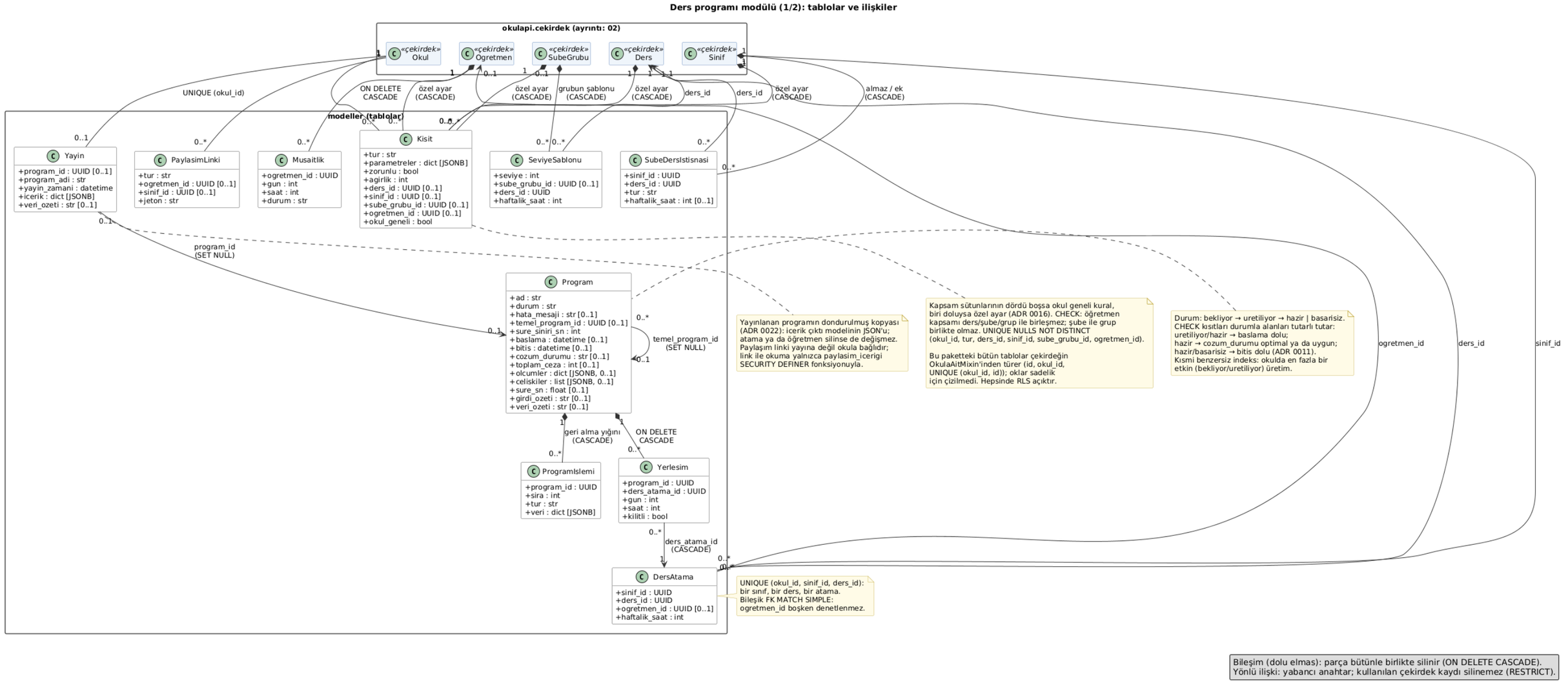
02 Sınıf diyagramı: Çekirdek modeller ve okul izolasyonu



Çekirdek tabloların ayrıntısı ve okula ait her tablonun ortak sütunlarını veren `OkulaAitMixin`. `kiraci` modülü okul izolasyonunun tek uygulama noktasıdır: her ORM sorgusuna okul koşulunu ekler, yeni kayda `okul_id` yazar ve işlem başında `SET LOCAL app.okul_id` ile PostgreSQL satır düzeyi güvenliğine (RLS) okulu bildirir. Giriş denemesi sayacı okula ait olmayan tek tablodur.

02-cekirdek-modeller.png · kaynak: docs/diyagramlar/02-cekirdek-modeller.puml

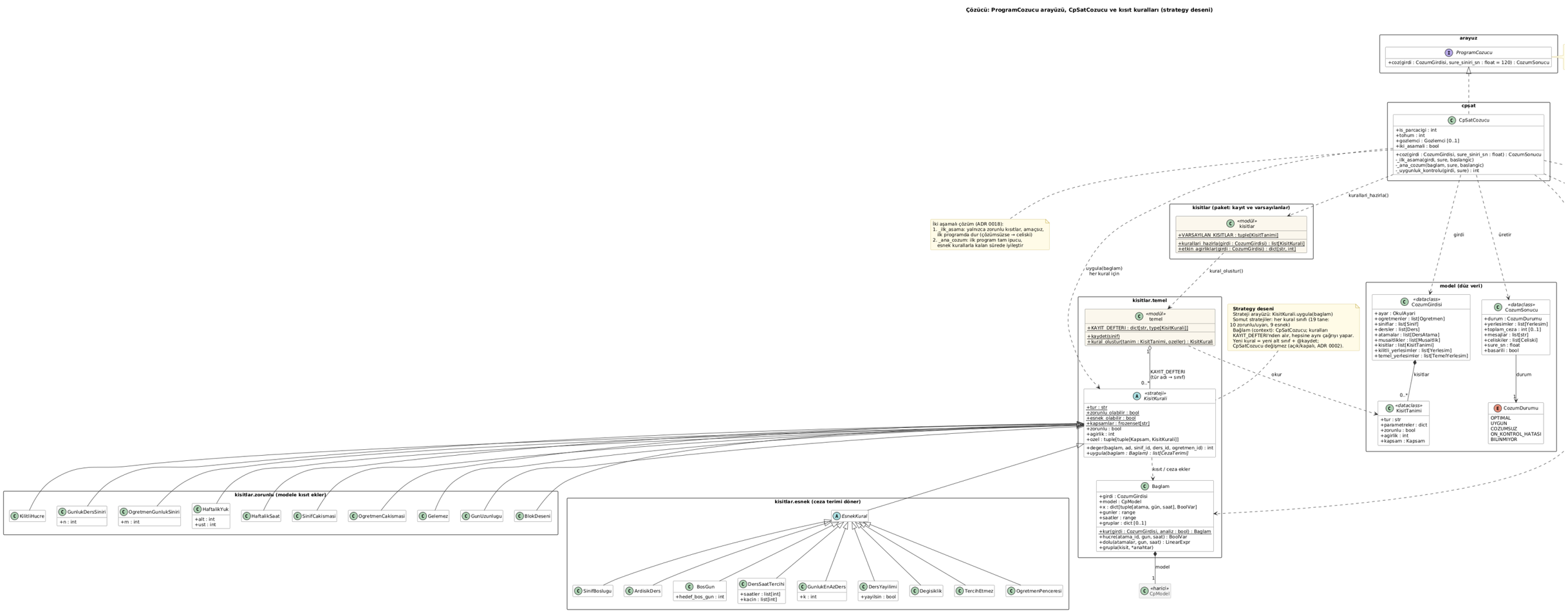
04 Sınıf diyagramı: Ders programı modülünün tabloları



Modülün tabloları, yabancı anahtarları ve silme davranışı (bileşim = `ON DELETE CASCADE`). Bütün ilişkiler bileşik yabancı anahtardır (`okul_id`, `x_id`): bir okulun satırı başka okulun satırını gösteremez. `Program` durum makinesi CHECK kısıtlarıyla, "okulda tek etkin üretim" kuralı kısmi benzersiz indeksle veritabanında da korunur.

04-ders-programi-tablolar.png · kaynak: docs/diyagramlar/04-ders-programi-tablolar.puml

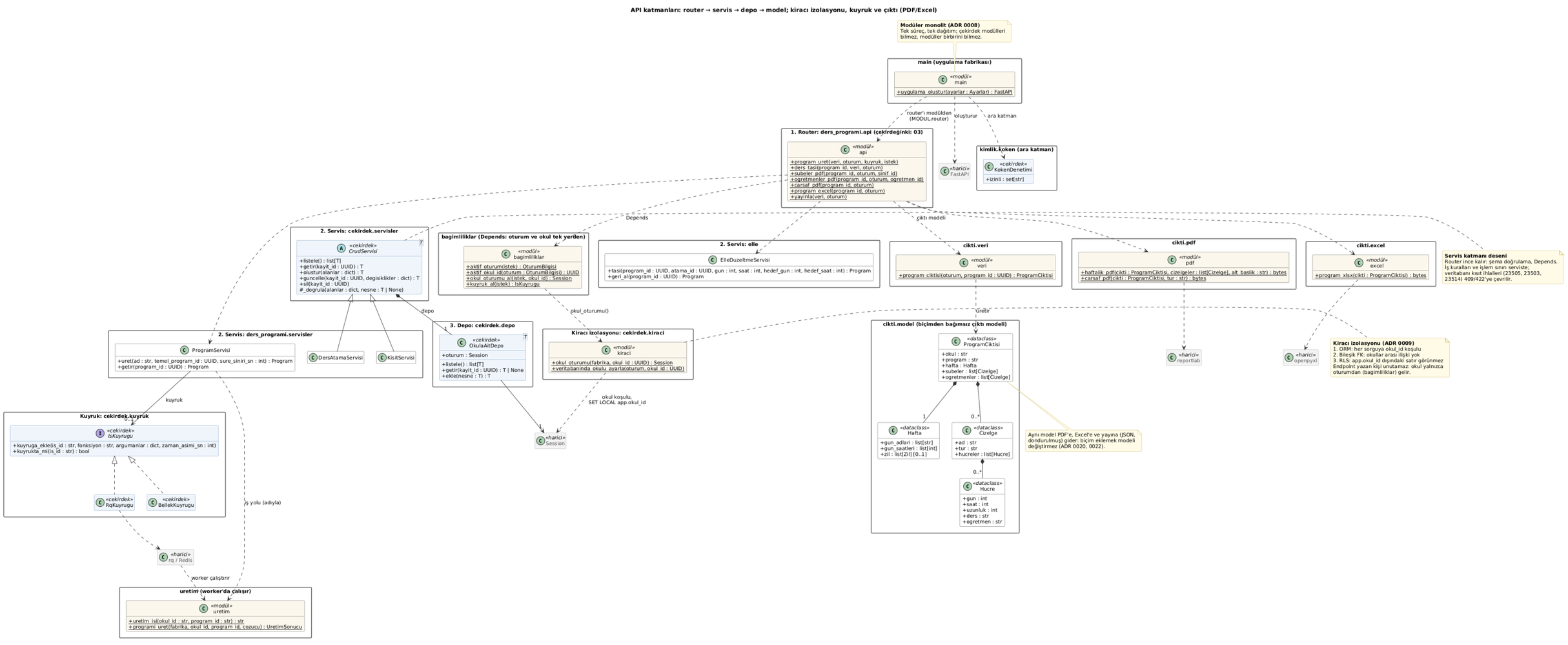
06 Sınıf diyagramı: Çözücü



`ProgramCozucu` arayüzü ve tek gerçeklemesi `CpSatCozucu`; kurallar strategy desenindedir: her zorunlu ya da esnek kural `KisitKurali`'ndan türeyen bir sınıftır ve kayıt defterine `@kaydet` ile girer, çözücü hepsine aynı `uygula(baglam)` çağrısını yapar. Ön kontrol, çelişki analizi, bağımsız doğrulama ve kalite ölçümü çözücü sınıfından ayrı modüllerdir; doğrulama ve kalite, çözücünün kendi kodunu kullanmayan ikinci bir görüştür.

06-cozucu.png · kaynak: docs/diyagramlar/06-cozucu.puml

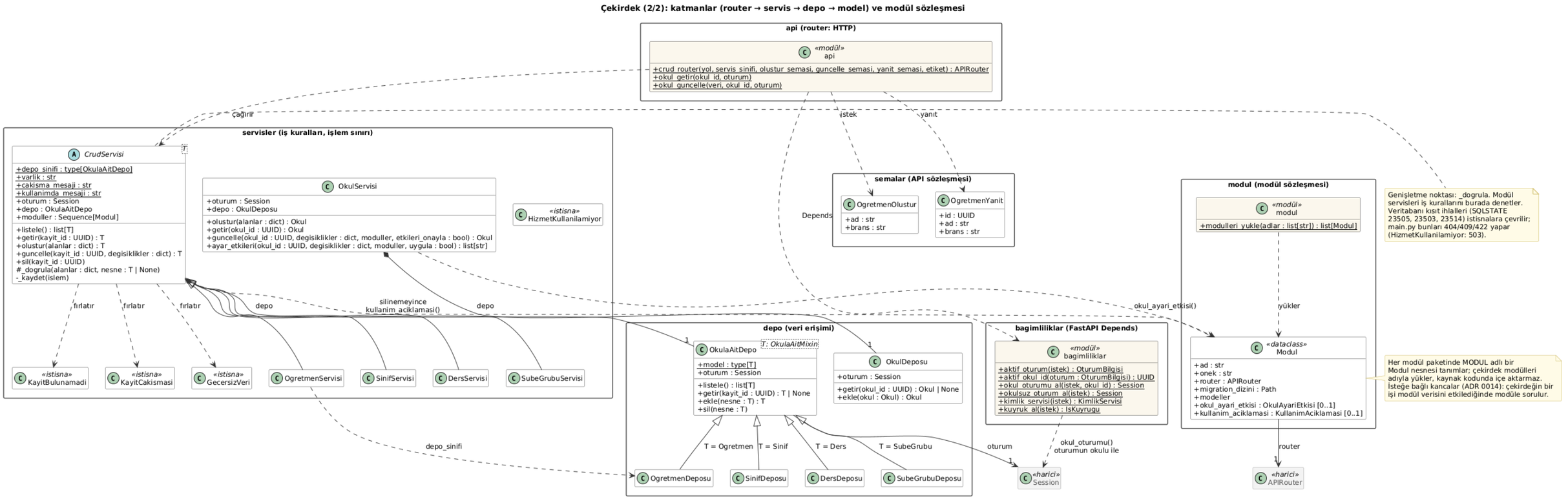
12 Sınıf diyagramı: API katmanları, kuyruk ve çıktı



İstek, router → servis → depo → model katmanlarından geçer; okul yalnızca oturumdan (`bagimliliklar`) belirlenir ve `kiraci` her sorguya uygulanır. Program üretimi servis tarafından `IsKuyrugu` arayüzüne konur (RQ gerçeklemesi, testlerde bellek ikizi) ve worker'da çalışır. PDF, Excel ve yayın, biçimden bağımsız tek bir çıktı modelinden (`ProgramCiktisi`) üretilir.

12-api-katmanlari.png · kaynak: docs/diyagramlar/12-api-katmanlari.puml

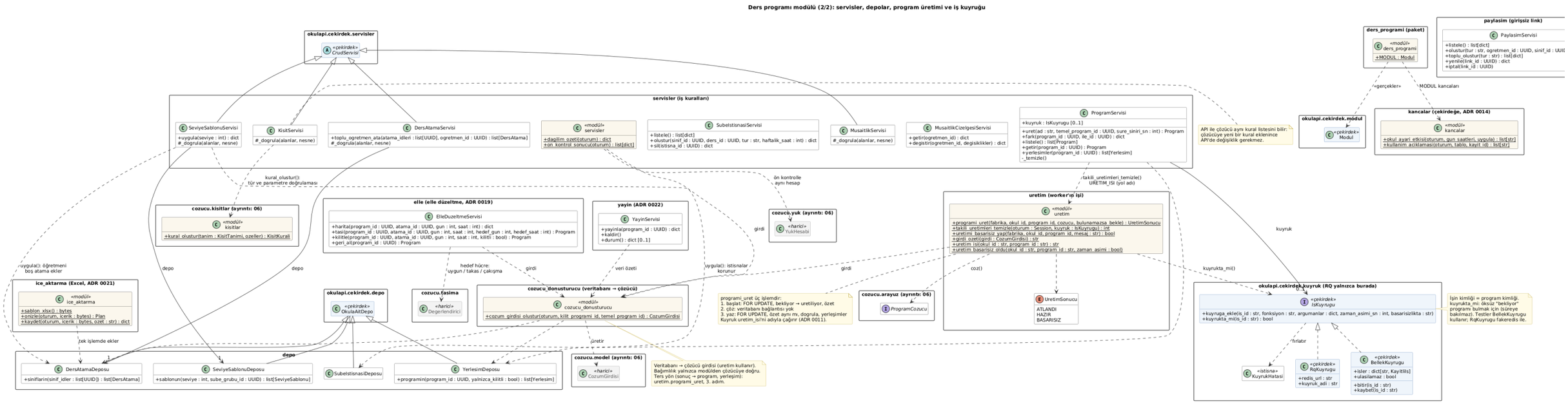
03 Sınıf diyagramı: Çekirdek katmanları ve modül sözleşmesi



Çekirdeğin router, servis ve depo katmanları: ortak CRUD davranışı `CrudServisi` ve `OkulaAitDepo` genel (generic) sınıflarındadır, alt sınıflar yalnızca iş kuralını (`_dogrula`) ekler. Modüller çekirdeğe `Modul` sözleşmesiyle kaydolur; çekirdek modülleri adıyla yükler ve kaynak kodunda içe aktarmaz.

03-cekirdek-katmanlar.png · kaynak: docs/diyagramlar/03-cekirdek-katmanlar.puml

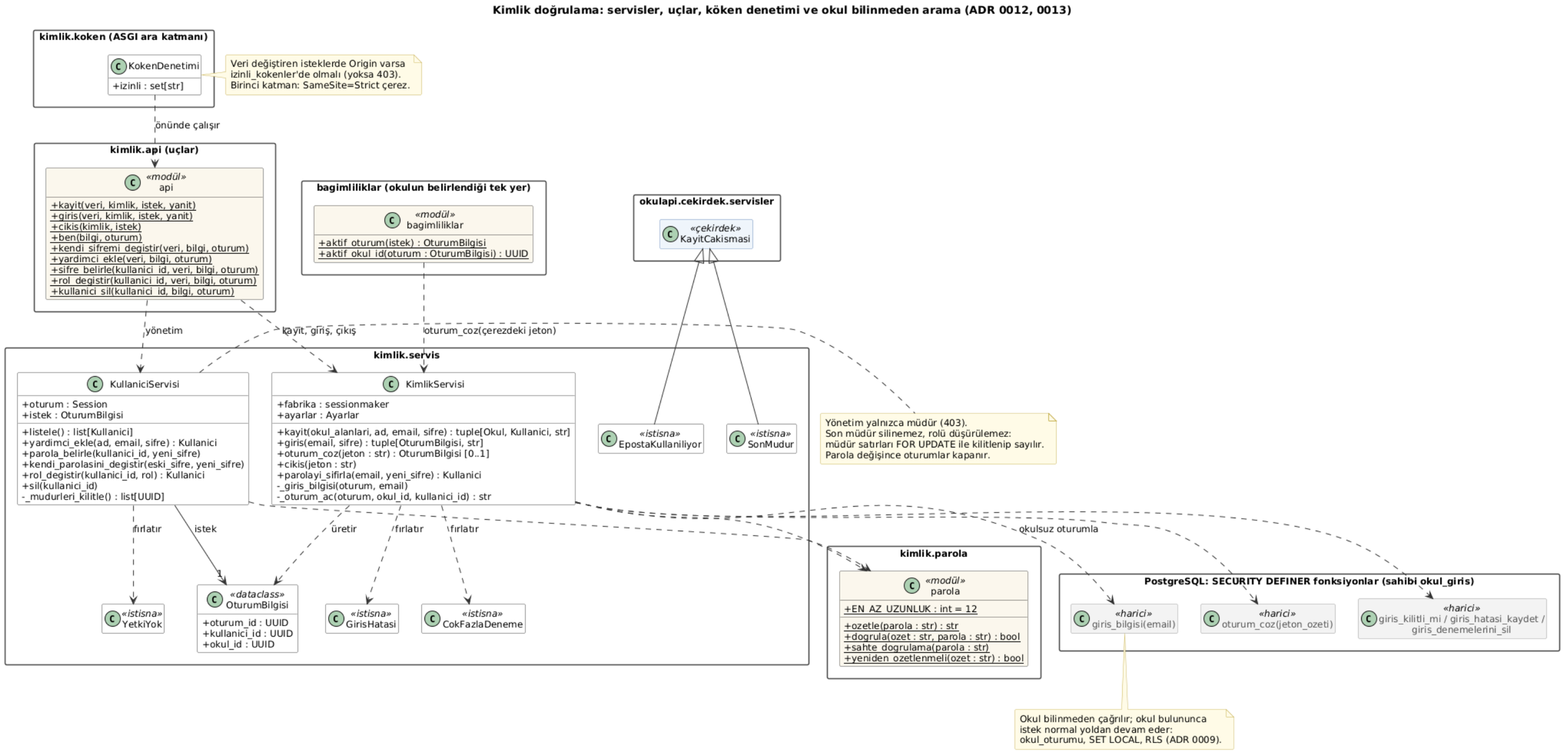
05 Sınıf diyagramı: Ders programı modülünün katmanları



Modülün servisleri (şablon, istisna, atama, kural, program, elle düzeltme, yayın, paylaşım, içe aktarma), depoları ve üretim işi. Veritabanından çözücüye geçiş yalnızca `cozucu_donusturucu` üzerindendir; bağımlılık hep modülden çözücüye doğrudur. `uretim.programi_uret` üç ayrı işlemdir (başlat, veritabanı bağlantısı olmadan çöz, yaz).

05-ders-programi-katmanlar.png · kaynak: docs/diyagramlar/05-ders-programi-katmanlar.puml

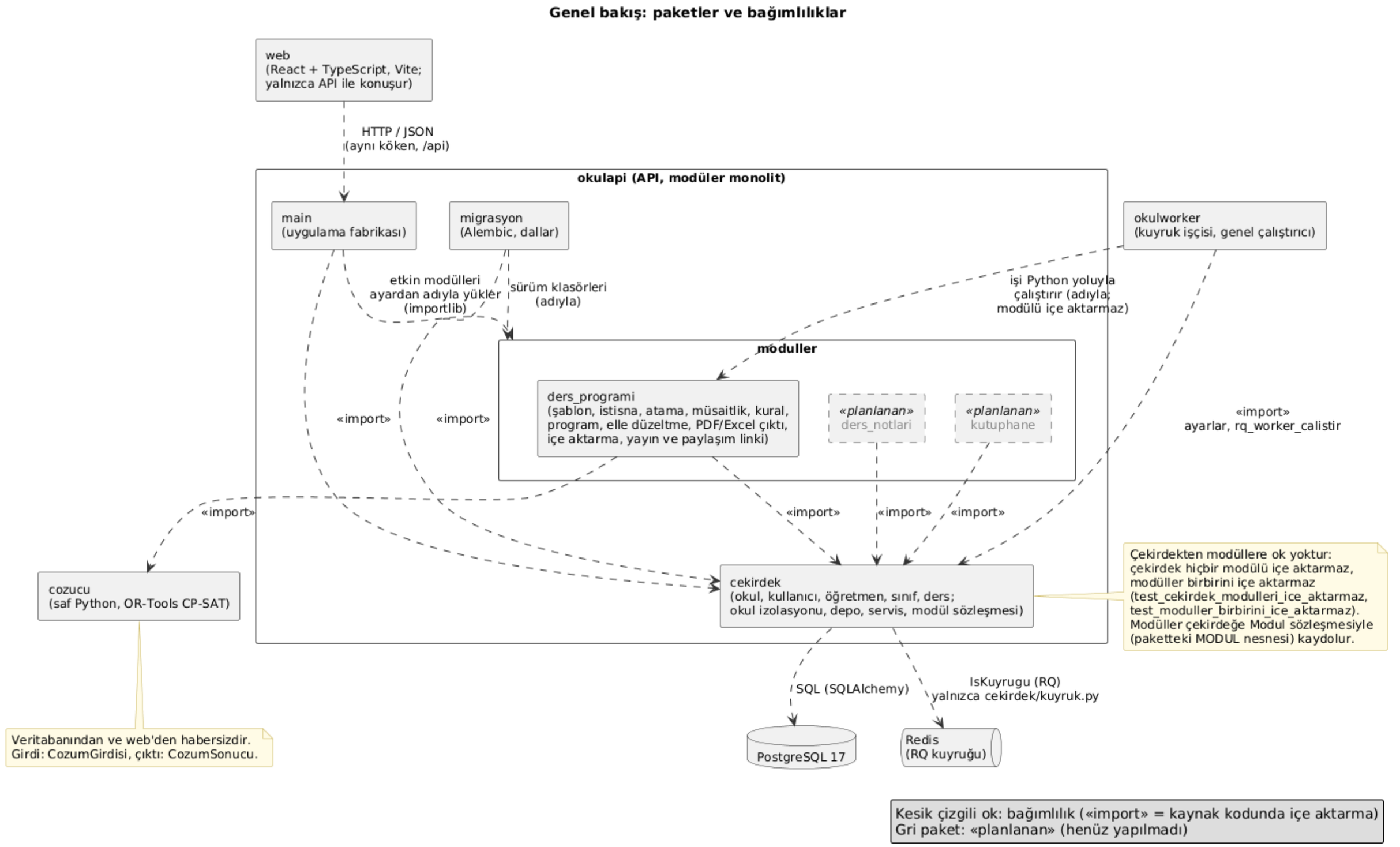
10 Sınıf diyagramı: Kimlik doğrulama



Kayıt, giriş, oturum ve kullanıcı yönetimi. Okul henüz bilinmediği için kullanıcı ve oturum, `okul_giris` rolüne ait dar kapsamlı `SECURITY DEFINER` fonksiyonlarla aranır; okul bulunduktan sonra istek normal yoldan (RLS altında) devam eder. Köken denetimi ve `SameSite=Strict` çerez CSRF'ye karşı iki katmandır.

10-kimlik.png · kaynak: docs/diyagramlar/10-kimlik.puml

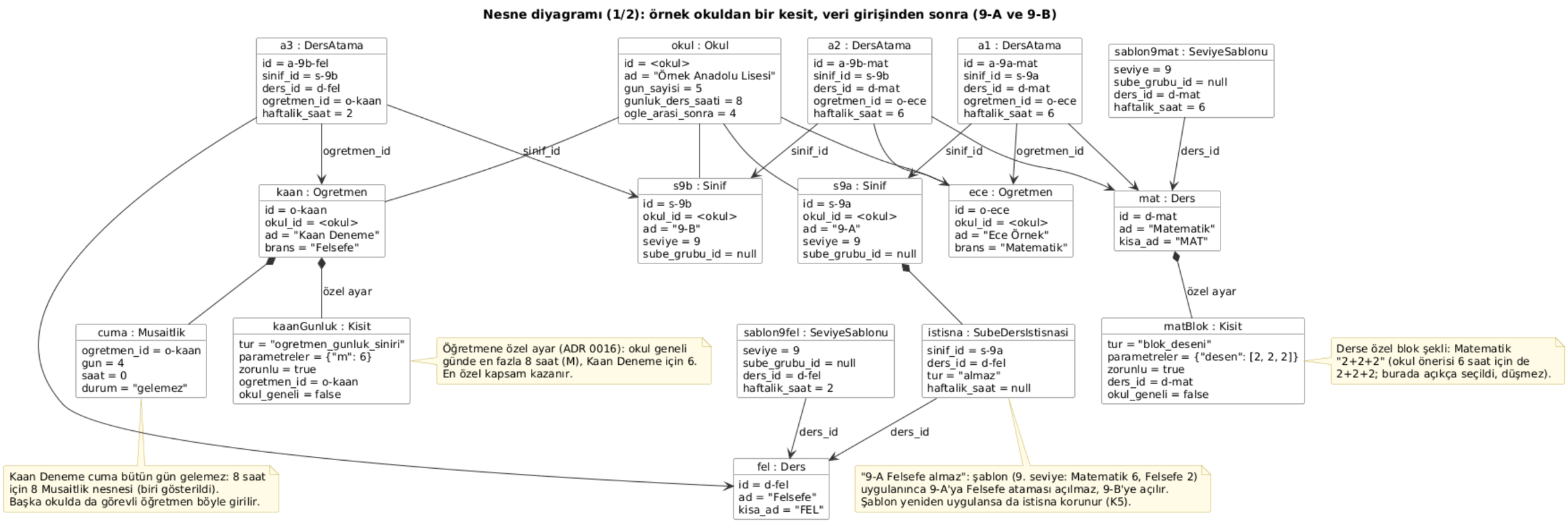
01 Paket diyagramı: Genel bakış



Çalışan parçalar ve paket bağımlılıkları. API modüler bir monolittir: çekirdek modülleri bilmez, modüller birbirini bilmez (testle denetlenir); çözücü veritabanından ve web'den habersiz saf Python paketidir. Worker modülü içe aktarmaz, işi Python yoluyla adından çalıştırır.

01-genel-bakis.png · kaynak: docs/diyagramlar/01-genel-bakis.puml

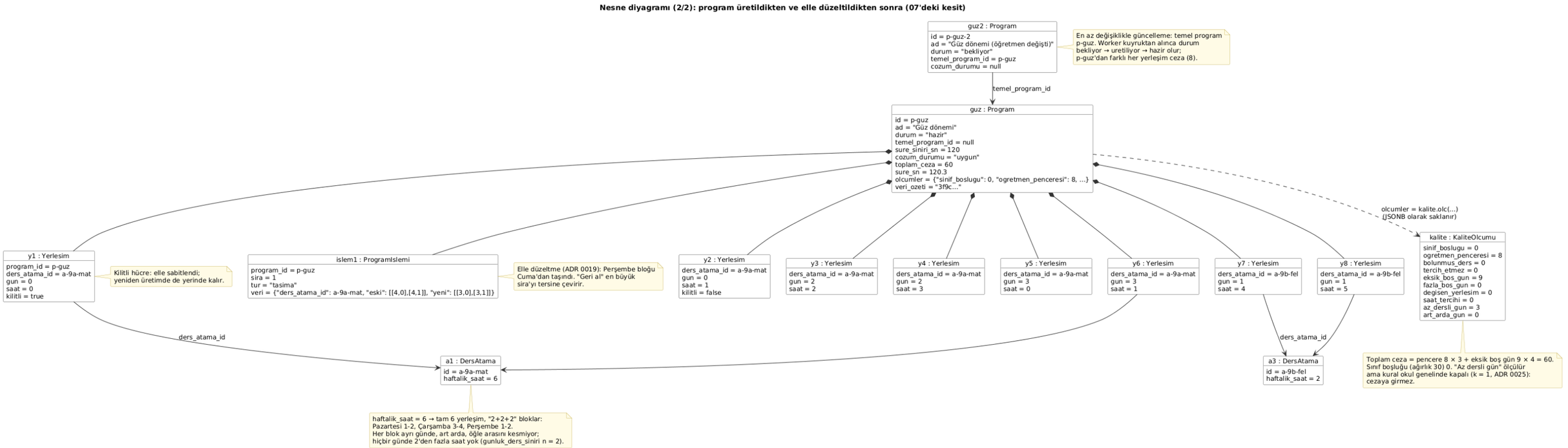
07 Nesne diyagramı: Örnek okuldan bir kesit (veri girişi)



İki şube (9-A, 9-B), iki öğretmen ve iki dersle veri girişinden sonraki anlık görüntü. 9. seviye şablonunda Felsefe olduğu hâlde "9-A Felsefe almaz" istisnası yüzünden yalnızca 9-B'nin Felsefe ataması vardır; bir öğretmene özel günlük sınır ve derse özel blok şekli de özel ayar (`Kisit` + kapsam) olarak görünür. Kişi adları uydurmadır.

07-nesne-veri-girisi.png · kaynak: docs/diyagramlar/07-nesne-veri-girisi.puml

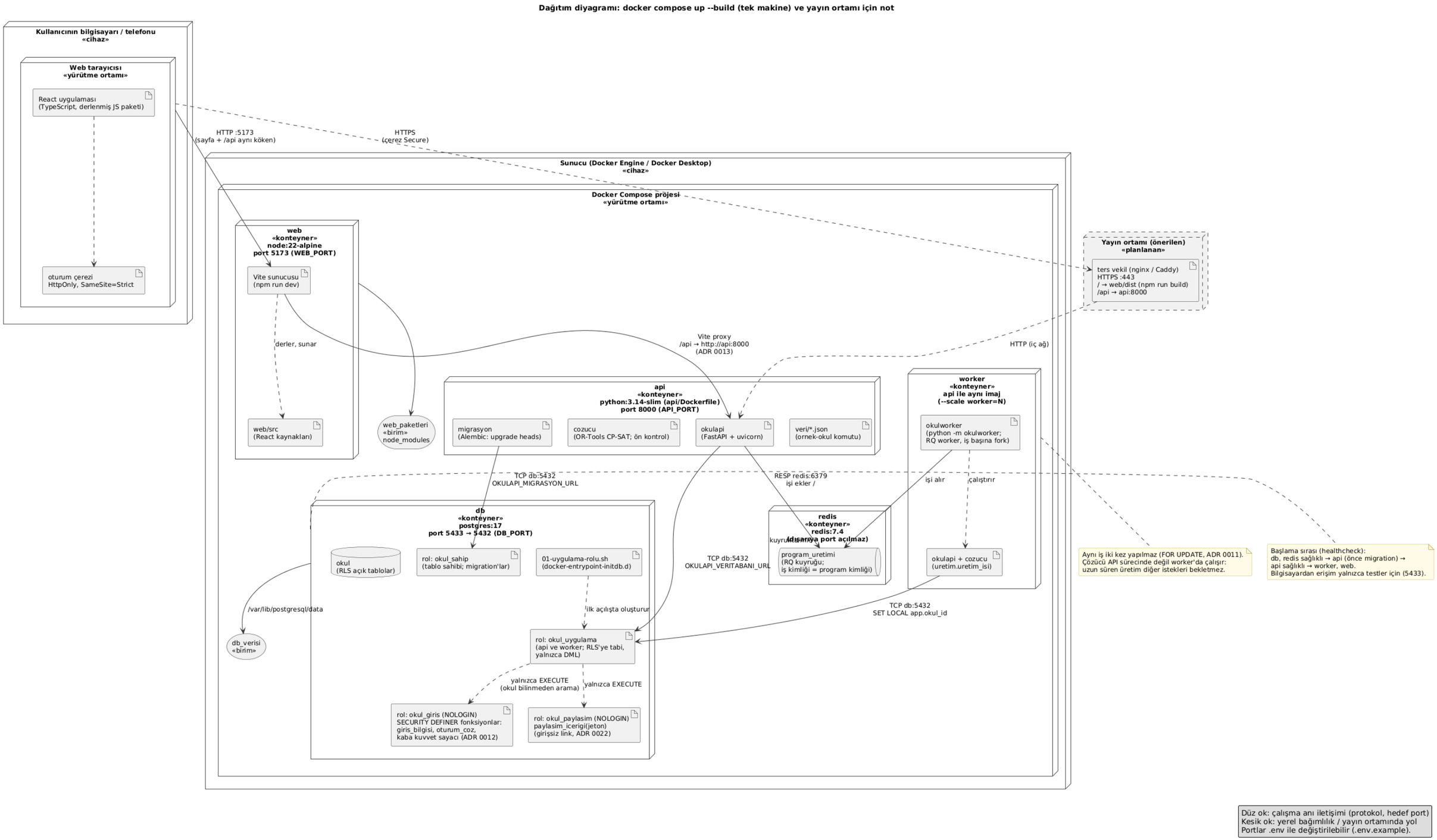
08 Nesne diyagramı: Program üretildikten sonra



Aynı kesit için hazır bir program: 9-A Matematik'in "2+2+2" bloklu altı yerleşimi (biri kilitli), geri alınabilir bir taşıma işlemi ve kalite özeti. Kalite değerleri gerçekçi setin 120 saniyelik bir ölçümünden alınmıştır (toplam ceza 60 = pencere 8 × 3 + eksik boş gün 9 × 4). İkinci program, ilkini temel alan en az değişiklikle güncelleme isteğidir.

08-nesne-program-sonrasi.png · kaynak: docs/diyagramlar/08-nesne-program-sonrasi.puml

09 Dağıtım diyagramı: Docker Compose



Beş konteyner (web, api, worker, redis, db), birimler, portlar ve protokoller. Tarayıcı yalnızca web kökenini görür; Vite `/api` isteklerini api konteynerine iletir (aynı köken, CORS yok). Uygulama ve worker RLS'ye tabi `okul_uygulama` rolüyle, migration'lar tablo sahibi rolle bağlanır; okul bilinmeden yapılan aramalar `NOLOGIN` rollerin fonksiyonlarıyladır. Yayında web derlenip bir ters vekil sunucunun arkasına HTTPS ile konur.

09-dagitim.png · kaynak: docs/diyagramlar/09-dagitim.puml